

Golden Arduino

Sarthak Samal^{a,1}

^aStudent

Dr. Eric Bogatin

Abstract—The Arduino UNO from the factory is a fully functional micro-controller out of the box, however, it lacks a feature we’ve come to expect from well-designed boards: The return plane. This lab takes up the challenge of recreating the Arduino with a single, continuous return plane. By the end of this lab, we successfully created a board that not only functioned as an Arduino board but improved upon its signal integrity, additionally, we learned about how the Arduino communicates with the IDE and peripherals.

Contents

1	Introduction	1
2	Objectives	1
3	Plan of Record	1
3.1	What does it mean to work?	1
3.2	Features of the Board	1
3.3	Risk Management	1
3.4	Key Measurements	2
4	Components	2
5	SBB Version of Arduino	2
6	Desinging the PCB	3
7	Assembling the PCB	4
8	Burning the Bootloader	4
9	Communicating with the IDE	5
10	Key Measurements: Power	6
11	Key Measurements: Data Lines	7
11.1	USB D+ and D- Lines	7
11.2	SDA and SCL Lines	7
11.3	Tx and Rx Lines	7
12	Testing Switching Noise	8
12.1	Commercial Arduino	9
	<i>Slammer Circuit • Microcontroller as the Aggressor</i>	
12.2	Golden Arduino	11
	<i>Slammer Circuit • Microcontroller as the Aggressor</i>	
12.3	Analysis	13
13	Conclusion	13

1. Introduction

While the Arduino developer board that comes packaged from the market works as intended, it is not built to be the best it could be in terms of signal integrity. One of the key features of a well-designed PCB is missing: the ground return plane. This is good for cheap board production for mass production, however, it presents a challenge for PCB designers: improve upon the board design by adding a return plane and using best design practices. We can also take this opportunity to see exactly how the Arduino (and consequently most other I²C components) work.

2. Objectives

To build the Golden Arduino, we will refer to the best design practices learned in prior labs and confirm it’s functionality by taking key measurements (specific measurements will be discussed in the plan of record). We will also compare the Golden Arduino against the commercial Arduino to determine if our board is actually better.

1. Create a solderless breadboard (SBB) of an Arduino. 17
2. Design the schematic and PCB in Altium Designer. 18
3. Construct the board. 19
4. Bootload the Golden Arduino and confirm the bootload. 20
5. Run a basic blink sketch and confirm basic functionality 21
6. Recreate the circuit and results for Lab 16 (differential and single-ended measurements) to confirm I²C functionality 22
7. Use a switching noise shield to try and induce noise on the MOSFET power die and examine the noise in both Arduinos 24

3. Plan of Record

3.1. What does it mean to work?

The functionality of the Golden Arduino will be confirmed by setting the following criteria:

1. Arduino must receive 5v from both the USB and power jack without complication. 30
2. All components of the Arduino must be powered by the 3.3v line from the LDO. 31
3. Arduino must be able to bootload and be recognized by the Arduino IDE. 32
4. All power lines must experience little to no noise during operation. 33
5. Default programs provided by Arduino LLC should run on the golden Arduino. 34
6. Lab 16 results must be replicated using the golden Arduino. 35
7. Golden Arduino must be able to output data to the serial monitor. 36
8. SDA and SCL lines must be communicating with peripheral devices. 37
9. Golden Arduino must have a smaller amount of switching noise compared to the commercial Arduino. 38

3.2. Features of the Board

1. 2-layer board (signal and return) 47
2. 5v VCC by either USB 2.0 mini-B or 5v DC power jack. 48
3. 1206 components for easy soldering 49
4. Switches to control power to CH340 and ATmega328p for debugging 50
5. Ground pins for compatibility with noise shield. 51
6. SMD components for a more compact design. 52

3.3. Risk Management

1. Decoupling capacitors were added to reduce noise on power lines. 53
2. Test points were added on all major data lines (SDA, SCL, USB D+, USB D-, Tx, Rx)) 54
3. Test points were added on all power lines 55
4. Connectivity was checked between all pins in ICs to ensure no unexpected shorts. 56
5. Sub-circuits were grouped such that traces do not interfere with other sub-circuits. 57

6. Cross-unders were kept as few as possible to ensure one, continuous ground plane.
7. Pins are spaced such that breadboard components are able to connect directly to the golden Arduino.

3.4. Key Measurements

1. Tx line must have around 20ms of data transfer and around 75ms of lag before the next data transfer (will be further elaborated in the Tx section).
2. Rx line must send an "end communication" sequence to the CH340.
3. SCL line must have a clock frequency of 100kHz and the SDA line must show data transfer line.
4. USB D+ must show the address and data being transferred, D- must be the inverse of D+.
5. 16MHz crystal must oscillate at 16MHz.
6. 12MHz crystal must oscillate at 12MHz.
7. 5v must be measured on the 5v line.
8. 3.3v must be measured on the 3.3v line.
9. Switching noise must be less than the commercial Arduino.

4. Components

1. ATmega328p x1
2. CH340p x1
3. 16MHz crystal oscillator x1
4. 12MHz crystal oscillator x1
5. Push button x1
6. TVS Diode
7. 5v to 3.3v LDO x1
8. Switch x2
9. 3-pin switch x1
10. 5v power jack
11. USB 2.0 mini-B connector x1
12. Red LED x5
13. 1M Ω resistor x2
14. 10k Ω resistor x1
15. 1k Ω resistor x5
16. 1 Ω resistor x1
17. 22 μ F capacitor x7
18. 10 μ F capacitor x2
19. 1 μ F capacitor x1
20. 22pF capacitor x4
21. 10 μ H ferrite bead x1

5. SBB Version of Arduino

Before designing the schematic and PCB, we must understand a key component to making an Arduino: Burning the bootloader. The ATmega chip doesn't come with the bootloader out of the factory; it comes up dumb, so it is up to the user to burn it into the chip.

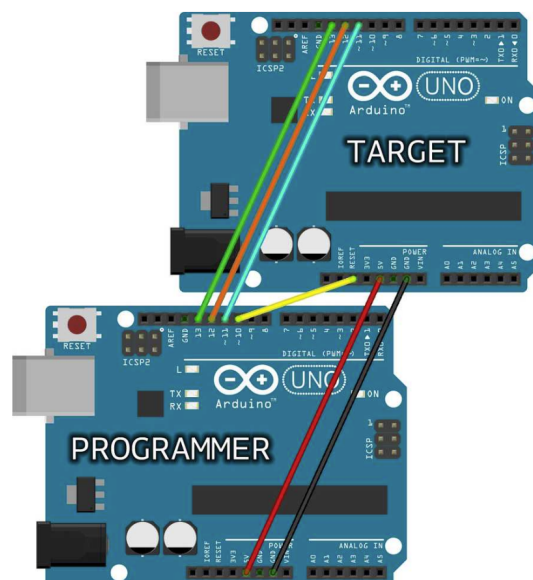


Figure 1. Connections to burn bootloader (image from Arduino LLC)

ATmega168 Pin Mapping			
Arduino function	ATmega168 Pin	ATmega168 Pin	Arduino function
reset	(PCINT14/RESET) PC6	26	PC5 (ADC5/SCL/PCINT13)
digital pin 0 (RX)	(PCINT16/RXD) PD0	27	PC4 (ADC4/SDA/PCINT12)
digital pin 1 (TX)	(PCINT17/TXD) PD1	28	PC3 (ADC3/PCINT11)
digital pin 2	(PCINT18/INT0) PD2	29	PC2 (ADC2/PCINT10)
digital pin 3 (PWM)	(PCINT19/OC2B/INT1) PD3	30	PC1 (ADC1/PCINT9)
digital pin 4	(PCINT20/XCK/T0) PD4	31	PC0 (ADC0/PCINT8)
VCC	VCC	32	GND
GND	GND	33	AREF
crystal	(PCINT6/XTAL1/TOSC1) PB6	34	AVCC
crystal	(PCINT7/XTAL2/TOSC2) PB7	35	PB5 (SCK/PCINT5)
digital pin 5 (PWM)	(PCINT21/OC0B/INT1) PD5	36	PB4 (MISO/PCINT4)
digital pin 6 (PWM)	(PCINT22/OC0A/AIN0) PD6	37	PB3 (MOSI/OC2A/PCINT3)
digital pin 7	(PCINT23/AIN1) PD7	38	PB2 (SS/OC1B/PCINT2)
digital pin 8	(PCINT0/CLKO/ICP1) PB0	39	PB1 (OC1A/PCINT1)

Digital Pins 11, 12 & 13 are used by the ICSP header for MOSI, MISO, SCK connections (ATmega168 pins 17, 18 & 19). Avoid low-impedance loads on these pins when using the ICSP header.

Figure 2. ATmega328p chip pinout (from Arduino LLC)

Luckily, Arduino LLC gives a detailed explanation of how to connect the programmer Arduino to the target Arduino (fig. 1) to burn the bootloader. With this and the ATmega328p chip pinout for reference (fig. 2), we can burn the bootloader to the chip, allowing it to communicate with the Arduino IDE.

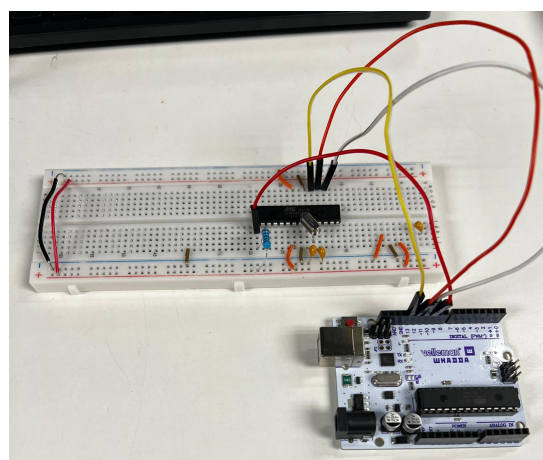


Figure 3. SSB bootloading set-up

Before burning the bootloader, we must add some components missing from the setup: the pullup resistor and 16MHz oscillating

crystal. We need the pullup resistor on the reset pin to end the communication between the programmer and target controllers and the 16MHz crystal sets a consistent clock for the target controller. From the TVS diode and crystal oscillator report, we found that adding two 22pF capacitors makes sure that the crystal does not oscillate at higher orders of frequency, therefore, we add them in this setup (fig. 3).

After these additions, we can successfully burn the bootloader. Before moving on, we can check to confirm if the chip is successfully burned. After burning, we can check the 16MHz crystal to make sure that it is oscillating at 16MHz, if so, we can be sure that the chip is successfully burned.

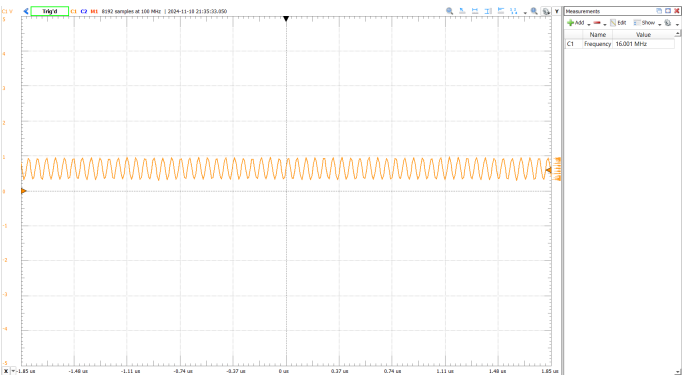
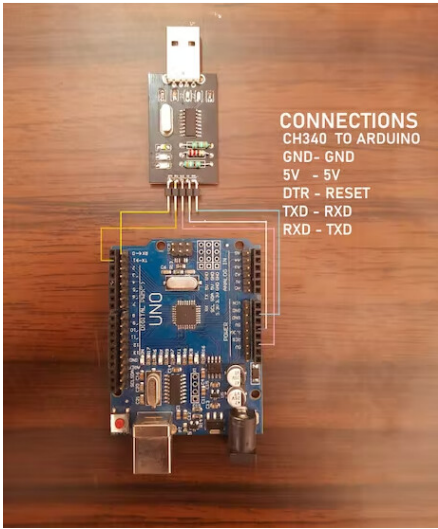
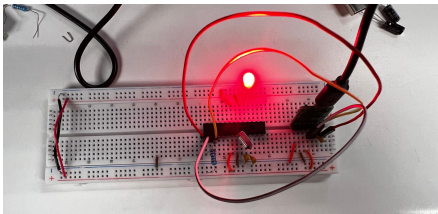


Figure 4. 16MHz crystal oscillation of SBB Arduino

After confirming the burn, we can introduce the CH340p module and ensure that the board can communicate with the Arduino IDE.



(a) Connection from CH340 to Arduino (from hackaday.io)



(b) SBB Arduino with blink program

Figure 5. CH340 connection to Arduino and SBB Arduino running blink program

After connecting the CH340 module to the ATmega328p chip (fig.

5a), we are able to connect the board to our computer and have it communicate with the IDE (fig. 5b). After successfully burning the bootloader and creating a SBB Arduino, we can move on to designing the schematic and PCB.

6. Desinging the PCB

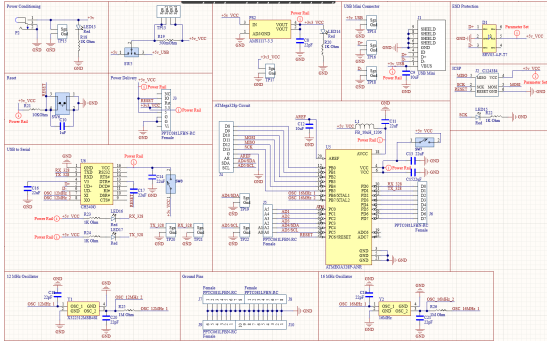
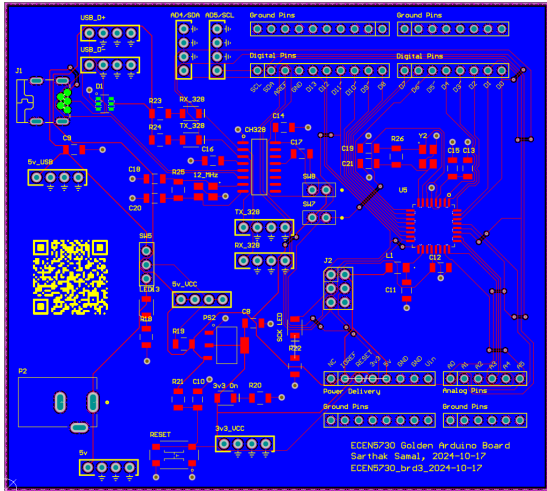
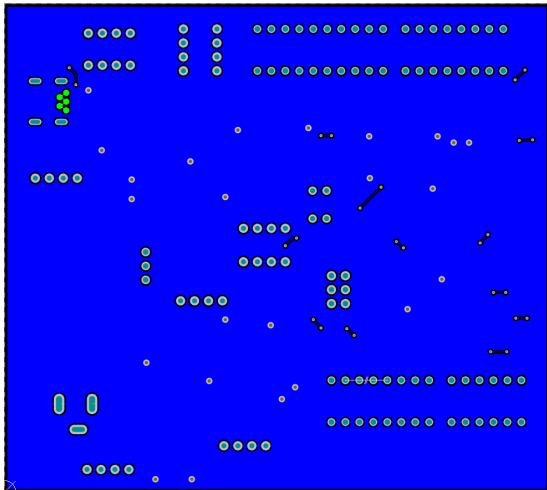


Figure 6. Caption



(a) PCB design of Golden Arduino (front)



(b) PCB design of Golden Arduino (back)

Figure 7. Schematic and PCB design of Golden Arduino

The biggest things in mind while making this board were as follows.

- 1. Keep sub-circuits close together

2. Minimize the amount of cross-unders needed
3. Maintain consistent spacing between the ground, analog, digital and power pins.

These will help to ensure simpler routing, which will help both signal integrity and debugging.

7. Assembling the PCB

Given the additional intricacies of this board and the emphasis on avoiding accidental shorts, the assembly process was a little more involved.

1. A magnifying glass was used to keep on eye on the soldering joints on the TVS diode and ATmega328p chip
2. The tip of the soldering iron was switched out for a thin tip to get in between the joints of the ATmega328p chip and TVS diode.
3. 70% Isopropyl alcohol was used to clean the board after soldering to ensure that flux residue doesn't hinder the function of the board.
4. After soldering a component, the connectivity between each pin was checked to ensure that there were no unexpected shorts.

With these measures, we can be confident that any soft or hard errors due to soldering can be avoided.

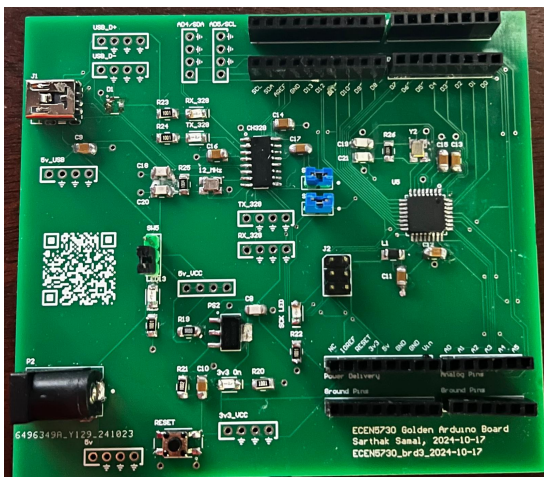
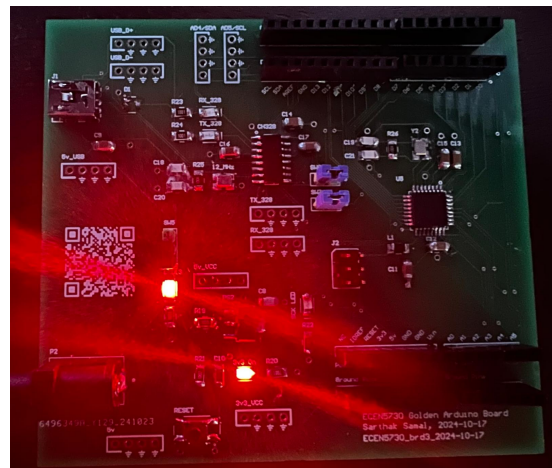


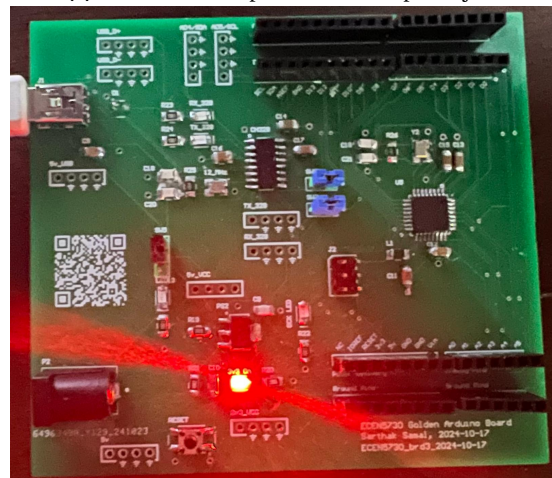
Figure 8. Assembled Golden Arduino

After taking proper measures to ensure that no parts will be damaged once powered, we can now power the board on and see if the respective power LEDs turn on.

Note: The TVS diode (next to the USB) is missing, this is due to a soldering mistake, and in trying to fix it, the SMD pad was removed. This is not detrimental to the function of the board, but some caution will be used to make sure there aren't any power surges.



(a) Golden Arduino powered with 5v power jack



(b) Golden Arduino powered with 5v from USB cable

Figure 9. Assembled Golden Arduino with different power sources

According to fig. 9, the board receives power from both the power jack and USB without any complications.

So far, there doesn't seem to be any immediate problems with the board. We can move on to burning the bootloader to the Golden Arduino.

8. Burning the Bootloader

To burn the bootloader, the same connections shown in fig. 1 were used for setup. After executing the burn, we can use the same technique used in the SBB version of the Arduino to make sure the board has been successfully burned: check the oscillation of the 16MHz crystal.

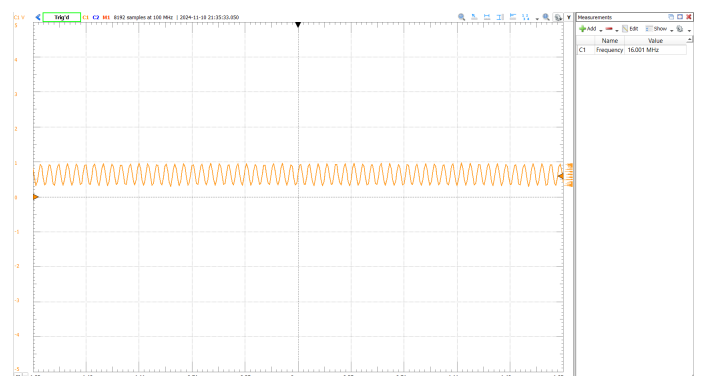


Figure 10. 16MHz crystal oscillation of Golden Arduino

According to fig. 10, the 16MHz crystal properly oscillates at 16MHz, indicating that the board was burned successfully. Similarly, we can check the oscillation of the 12MHz crystal connected to the CH340 chip to check for its functionality.



Figure 11. 12MHz crystal oscillation of Golden Arduino

From fig. 10 and 11, we can see that the board is successfully burned and the CH340 is working properly. Given this, we have good reason to suspect the board can communicate with the Arduino IDE.

9. Communicating with the IDE

To check for proper communication between the Golden Arduino and the IDE, we can follow these steps:

- 1. Check if the IDE recognizes the port
- 2. Check if the IDE can upload a basic "blink" example script
- 3. Check if the board can send data to the serial monitor

If the board checks of these criteria, it is safe to say that we have created a board capable of communicating with the IDE.

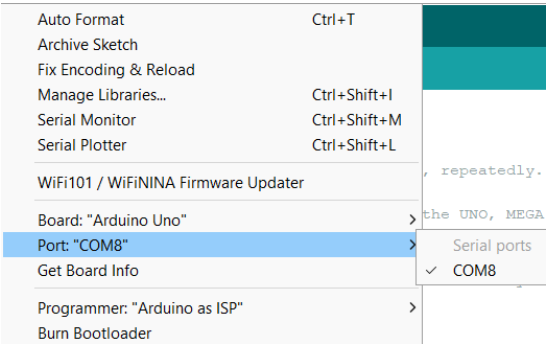


Figure 12. Port connection in the IDE

Upon connecting the board to the computer, the IDE immediately recognizes and connects to COM8 (fig. 12). After connection, we attempt to upload the "blink" program to the Arduino.

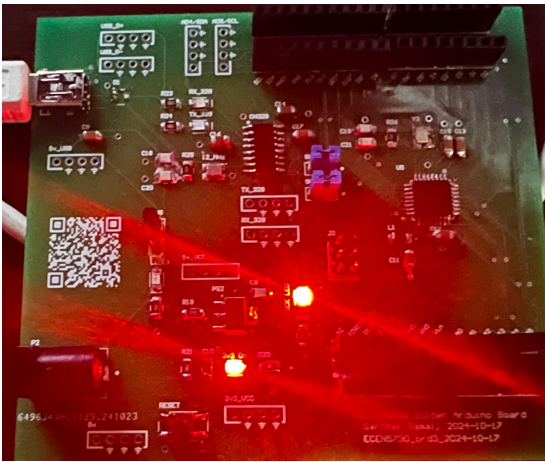


Figure 13. Golen Arduino running blink

According to fig. 13, the Golden Arduino is able to flash the SCK LED (i.e. the built-in LED the blink script uses)

Lastly, we need to check if the Arduino can send data to the serial monitor. For this, we can take the opportunity to use the code in our differential measurements lab.

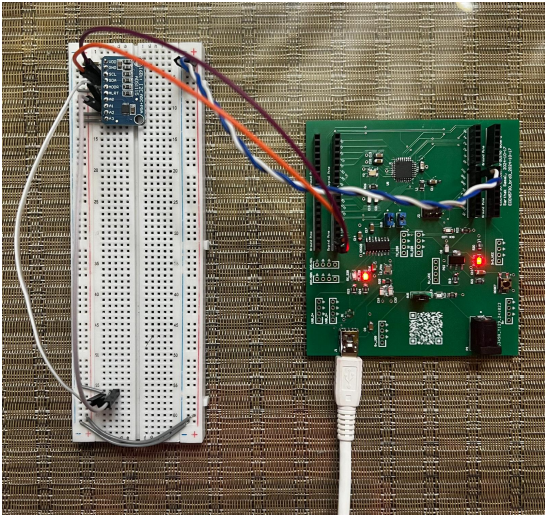


Figure 14. TMP setup with Golden Arduino

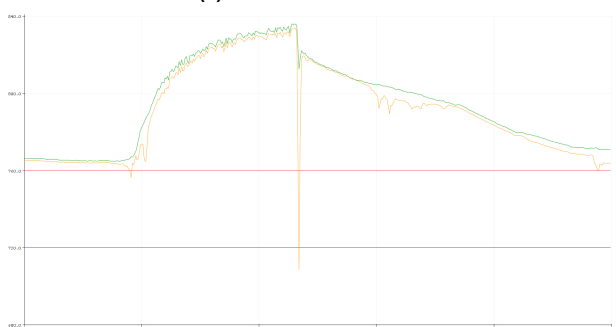
Constructing the setup in fig. 14 and uploading our code to take a differential measurement between A0 and A1 of the ADS, we can check if any results are sent to the serial monitor.

```

21:30:26.806 -> 720, 760, 727.80, 728.53
21:30:26.948 -> 720, 760, 727.89, 728.60
21:30:27.041 -> 720, 760, 728.24, 721.32
21:30:27.136 -> 720, 760, 728.44, 685.65
21:30:27.223 -> 720, 760, 727.83, 728.39
21:30:27.362 -> 720, 760, 727.73, 728.57
21:30:27.442 -> 720, 760, 727.74, 728.57
21:30:27.530 -> 720, 760, 727.70, 728.57
21:30:27.656 -> 720, 760, 727.71, 728.53
21:30:27.750 -> 720, 760, 727.83, 728.50
21:30:27.842 -> 720, 760, 727.80, 728.57
21:30:27.936 -> 720, 760, 727.75, 728.48
21:30:28.077 -> 720, 760, 727.79, 728.46
21:30:28.170 -> 720, 760, 727.82, 728.56
21:30:28.263 -> 720, 760, 727.69, 728.59

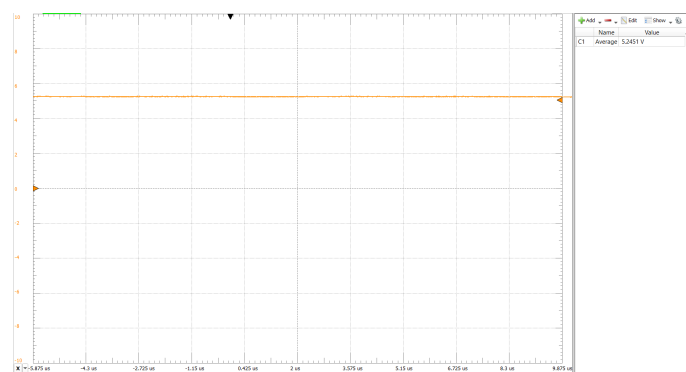
```

(a) Serial monitor on IDE



(b) Serial plotter on IDE

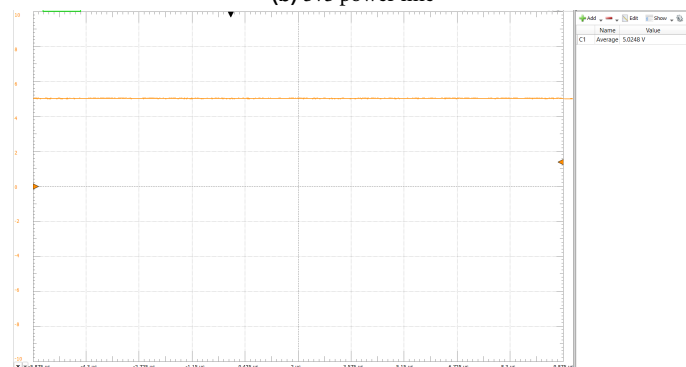
Figure 15. Serial outputs



(a) 5v power jack line



(b) 3v3 power line



(c) 5v USB line

Figure 16. Main power line readings

In fig. 16a, we see that the power jack is supplying a bit more than expected at 5.2v. However, this is within a tolerable range for our components and we still get 3.3v from the LDO (fig. 16b), so this is not an issue. The 5v from the USB is giving us 5v as expected (fig. 16c).

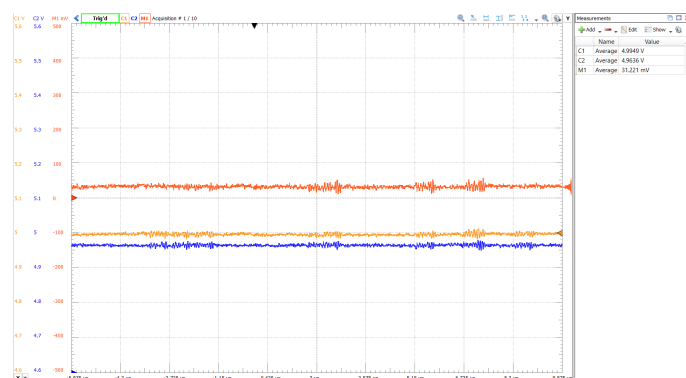


Figure 17. Differential measurement across sense resistor

Fig. 17 shows that the sense resistor gives us a readable voltage

across the resistor ($>20\text{mV}$) meaning that the resistor does not negatively influence the power line of the board. Allowing us to troubleshoot any issues on the power line should there be any.

Note: No significant in-rush transient was detected, therefore, no in-rush measurement was included.

Neither of the power lines experience voltage drops during operation, therefore, we can be confident that the board is sufficiently mitigating any noise.

11. Key Measurements: Data Lines

The data lines require a bit more of an explanation. Before each measurement, we will discuss the function of each data line to get an idea of what to expect. The program used for fig. 15 will be used for all of the data line measurements.

Note: All measurements will be shown in the same graph with one of the lines held at an offset for visibility.

11.1. USB D+ and D- Lines

The USB D+ and D- lines are responsible for the communication between the computer and the CH340 module. The data on these lines have 2 sections: The address to which data should be written (first 8 bits), and the data itself (the rest of the data).

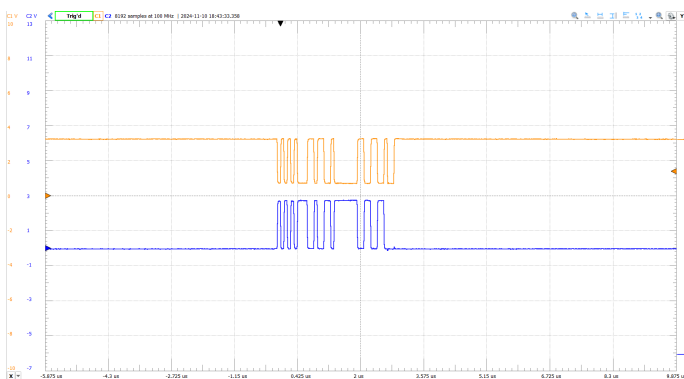


Figure 18. USB D+ (orange) and D- (blue) lines

In fig. 18 we can see bits of information traveling through the lines. Something that cannot clearly be seen is that the first 8 bits (address) remain the same while the rest of the bits (data) continuously change. With that, we can be sure that the D+ and D- lines work.

11.2. SDA and SCL Lines

The SDA and SCL lines are the heart of the I²C interface and are responsible for communicating with peripherals (in this case, the ADS). To verify the functionality of the board, we do not need to go in-depth on how I²C communication works, all we need to see is that the clock (SCL) must have a frequency of 100kHz (the default speed of I²C communications) and that some information transfer is visible on the SDA line.

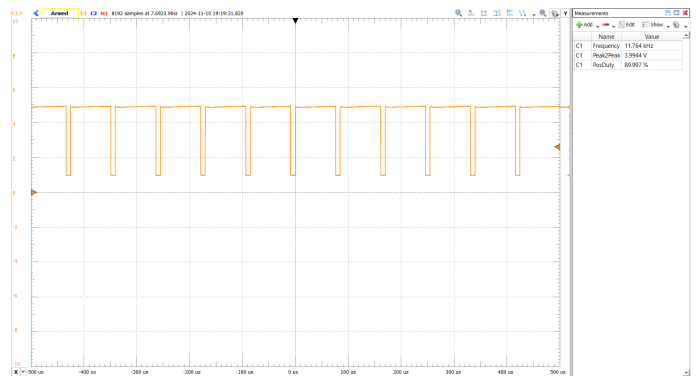


Figure 20. Tx reading on Golden Arduino

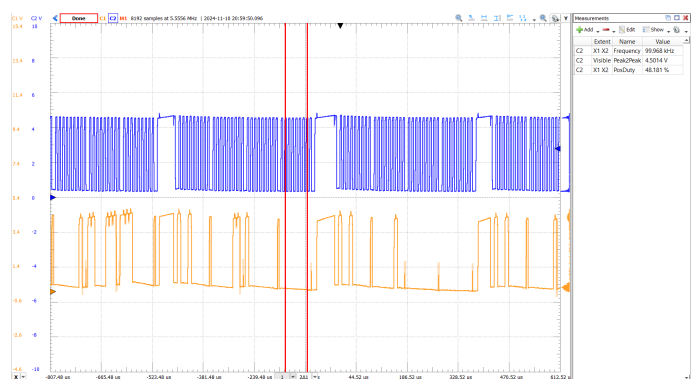


Figure 19. SDA (orange) and SCL (blue) measurements

According to fig. 19 we see that the SCL line has a frequency of 100kHz as expected with data transfer being apparent on the SDA line. It should be noted that the occasional gaps in the clock are due to the communication start and stop sequences between the controller (ATMega328p) and peripheral (ADS). From these readings, we can say that the SDA and SCL work as intended.

11.3. Tx and Rx Lines

The Tx and Rx lines are responsible for communication between the CH340 and ATMega328p and any serial monitor outputs. The Tx is responsible for transmitting data from the CH340 to the ATMega while the Rx is responsible for receiving data. We can compare the serial monitor outputs to the Tx reading to verify that the Tx works as expected.

From fig. 15a, we see that there is an average of about 70ms between each serial monitor output. Additionally, using a string-to-binary converter (fig. 21), we can see that we are transferring 200 bits of data per serial monitor output.

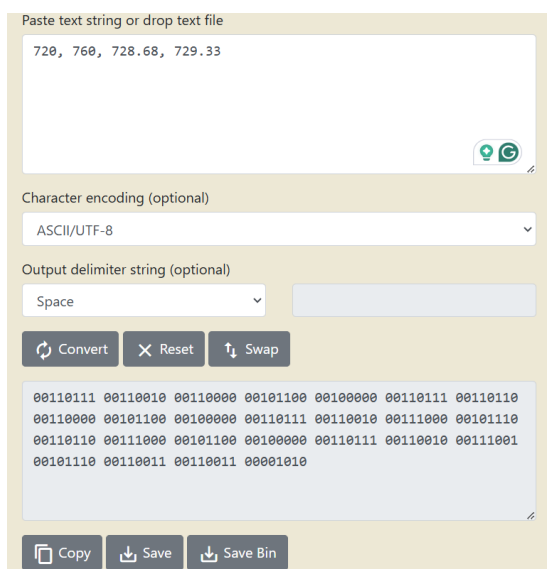
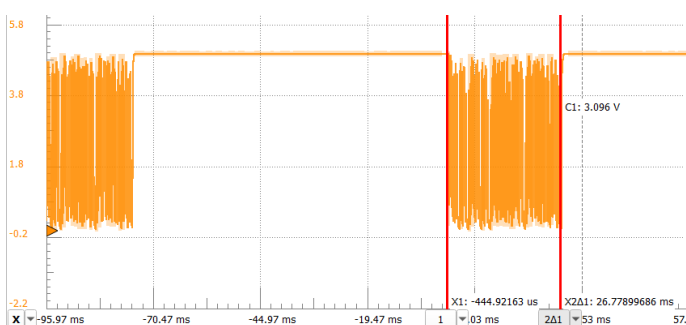
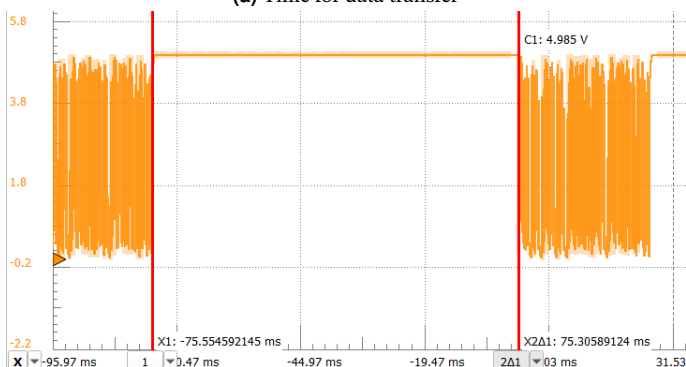


Figure 21. Converting one line of the serial monitor output to binary (from Rapidtables)

Since we are running the serial monitor at a baud rate of 9600, that means we are transferring 9600 bits of information per second. For 200 bits, that amounts to 20ms.



(a) Time for data transfer



(b) Time until next data transfer

Figure 22. Tx measurement breakdown

We see these calculations reflected in the Tx lines, with a data transfer time of about 26ms (fig. 22a) and a lag time of about 75ms (fig. 22b) until the next data transfer. With fig. 22 we can be certain that the Tx line works as intended.

The Rx line on the other hand is not as intuitive. To check the readings on this line, a normal trigger was used to capture the data being received when the blink program was being uploaded to the Golden Arduino.

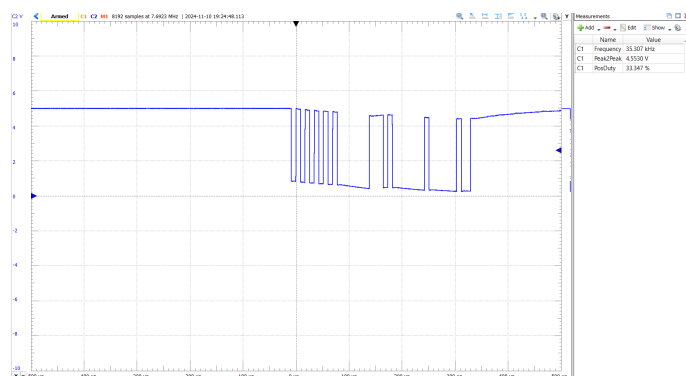


Figure 23. Rx reading

Since a normal trigger was used, the reading in fig. 29 is the tail-end of the data being received. Given that, it is likely to be a signal from the CH340 to indicate that communication has ended. In any case, getting a reading on the Rx line indicates that this line also functions as expected.

12. Testing Switching Noise

Now that we have confirmed a functional Golden Arduino, the next step is to test it against the commercial Arduino. To do this, we will use a switching noise shield, which will allow us to safely draw high currents from the Arduino to induce noise on the power lines.

We will first use the Arduino as the aggressor and begin switching an increasing number of pins and monitoring the noise induced in the Arduino power die.

Next, we will use a slammer circuit built into the shield to draw high currents from the MOSFET power die. The noise on this die will then be monitored.

Before comparison, we must take note of a soft error in the Golden Arduino. The switching shield is designed to fit directly onto the Arduino without any jumper wires. While it fits the commercial Arduino, it does not fit the Golden Arduino. Because of this, jumper wires will be used for both the commercial and Golden boards to make sure the results stay consistent.

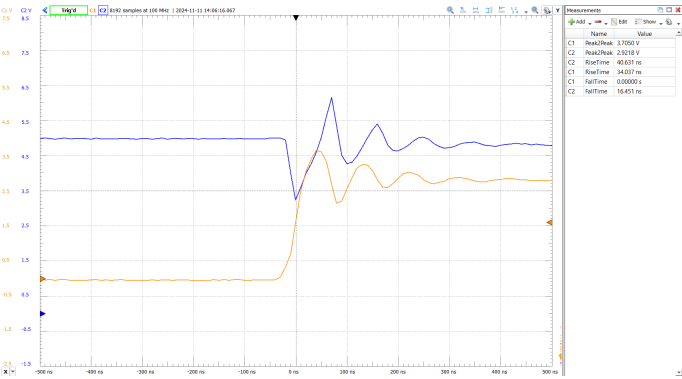
```
1 void setup() {
2   DDRB = B00111111;
3   pinMode(7, OUTPUT);
4   digitalWrite(7, LOW);
5 }
6 void loop() {
7   PORTB = B00111101;
8   delayMicroseconds(4);
9   PORTB = B00000001;
10  delay(1);
11  digitalWrite(7, HIGH);
12  delayMicroseconds(400);
13  digitalWrite(7, LOW);
14  delay(10);
15 }
```

Code 1. Driving Digital Pins

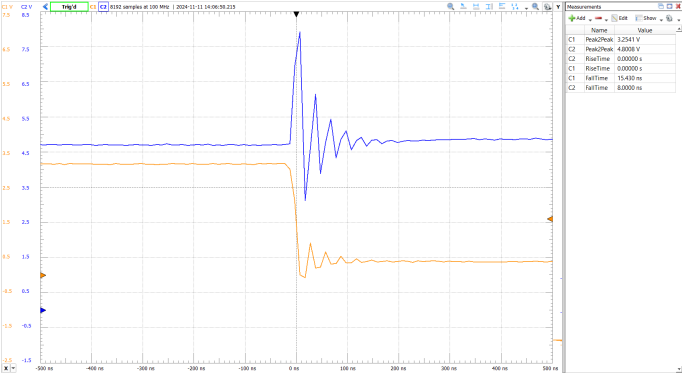
The above code shows the code we will upload to our Arduino to drive communications. Lines 7 to 10 will be commented out for the slammer circuit and 11 to 14 will be commented for the microcontroller aggressor loop.

12.1. Commercial Arduino

12.1.1. Slammer Circuit



(a) Rising edge of slammer circuit

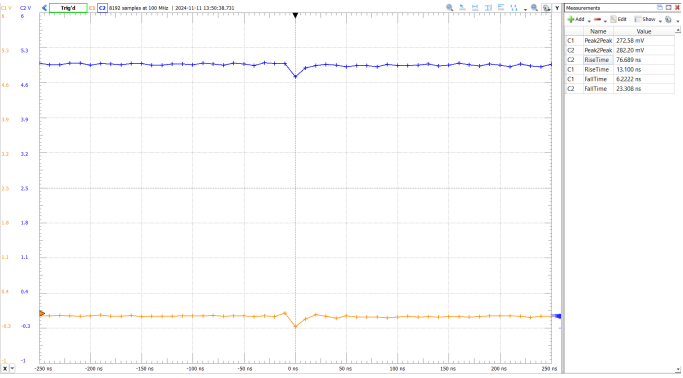


(b) Falling edge of slammer circuit

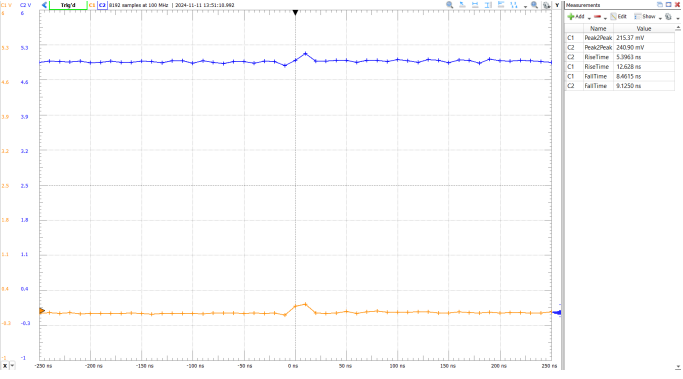
Figure 24. Noise induced (blue) on rising and falling edge of slammer signal (orange)

In fig. 30, we see that for the commercial Arduino, the power line has a p-p voltage of 3.7v and 2.9v on the rising and falling edge of the slammer signal, respectively.

12.1.2. Microcontroller as the Aggressor



(a) Noise on the rising edge of switch

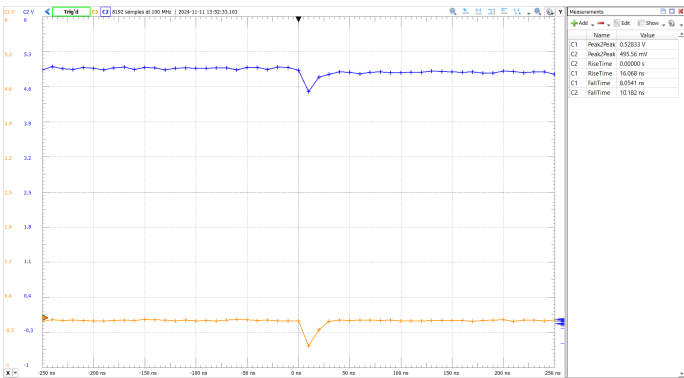


(b) Noise on the falling edge of switch

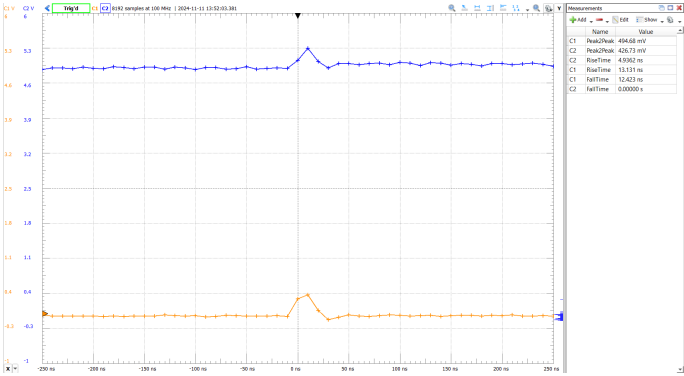
Figure 25. Noise induced on the HI (blue) and LO (orange) on the rising and falling edge of the switching signal (orange) with only pin 13 switching

Pin 13:

- 1. Rising Edge Vpp: .272v, .282v
- 2. Falling Edge Vpp: .215v, .240v

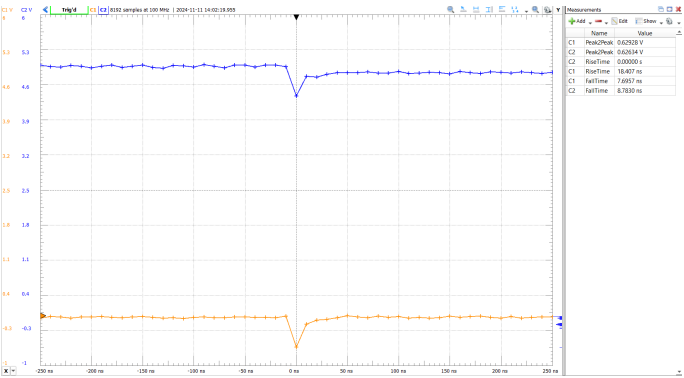


(a) Noise on the rising edge of switch

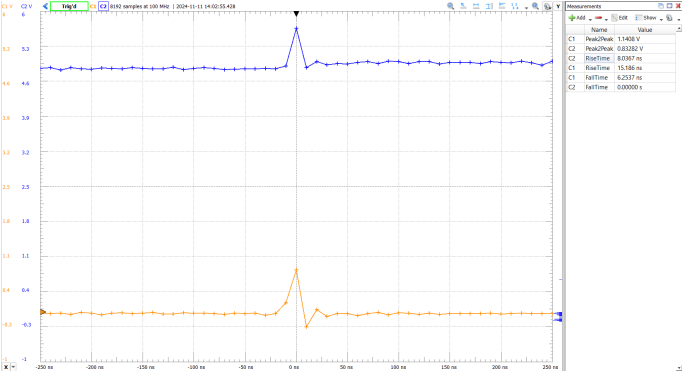


(b) Noise on the falling edge of switch

Figure 26. Noise induced on the HI (blue) and LO (orange) on the rising and falling edge of the switching signal (orange) with pins 13 and 12 switching



(a) Noise on the rising edge of switch



(b) Noise on the falling edge of switch

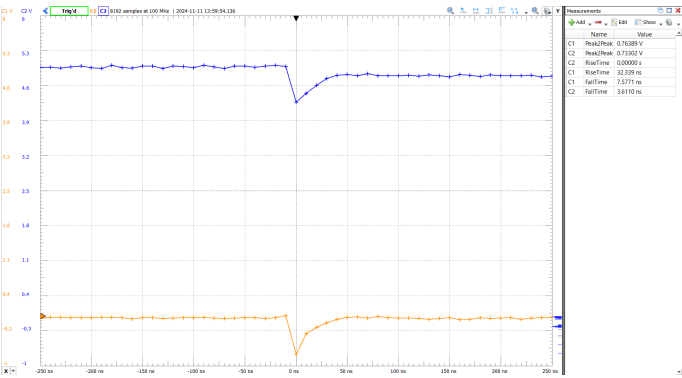
Figure 27. Noise induced on the HI (blue) and LO (orange) on the rising and falling edge of the switching signal (orange) with pins 13, 12, and 11 switching

Pins 13 and 12:

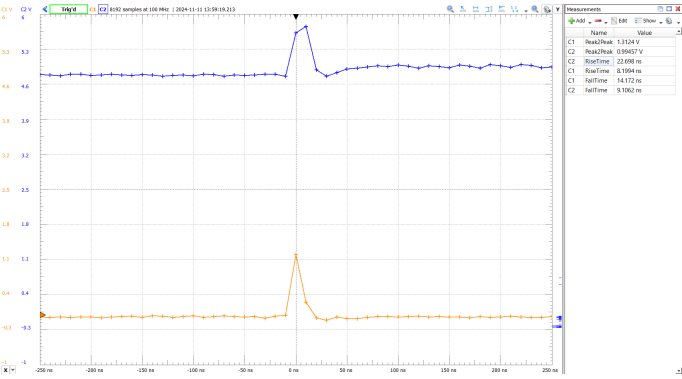
1. Rising Edge Vpp: .528v, .496v
2. Falling Edge Vpp: .495v, .427v

Pins 13, 12 and 11:

1. Rising Edge Vpp: .629v, .626v
2. Falling Edge Vpp: 1.14v, .838v



(a) Noise on the rising edge of switch



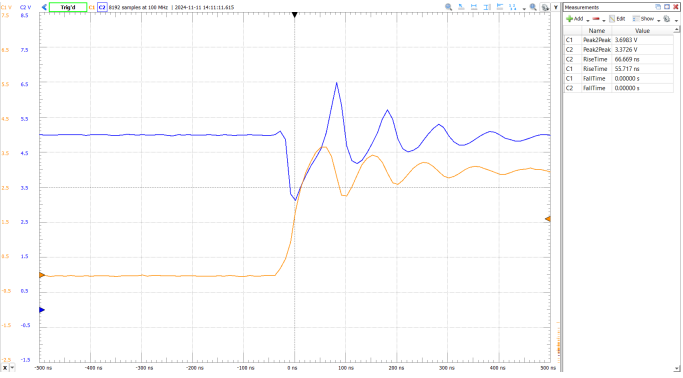
(b) Noise on the falling edge of switch

Figure 28. Noise induced on the HI (blue) and LO (orange) on the rising and falling edge of the switching signal (orange) with pins 13, 12, 11, and 10 switching

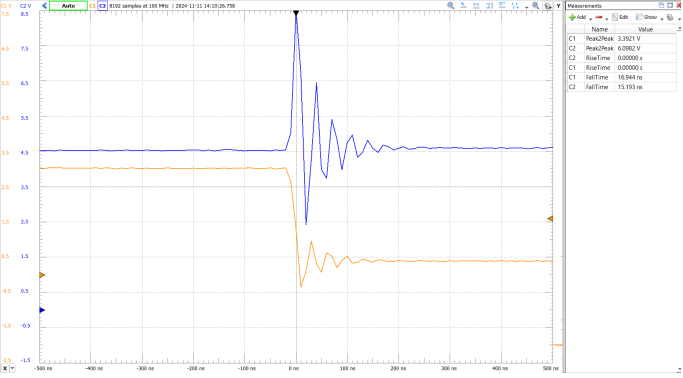
- Pins 13, 12, 11 and 10
1. Rising Edge Vpp: .76v, .73v
 2. Falling Edge Vpp: 1.31v, .99v

After completing the test, we see that as the number of switching pins increases, the noise on the HI and LO lines also increases.

12.2. Golden Arduino
12.2.1. Slammer Circuit



(a) Rising edge of slammer circuit

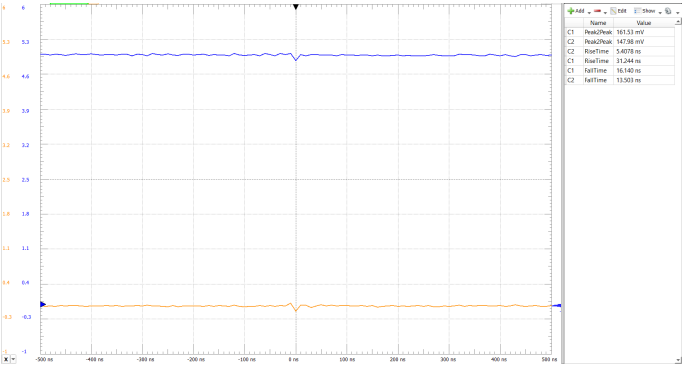


(b) Falling edge of slammer circuit

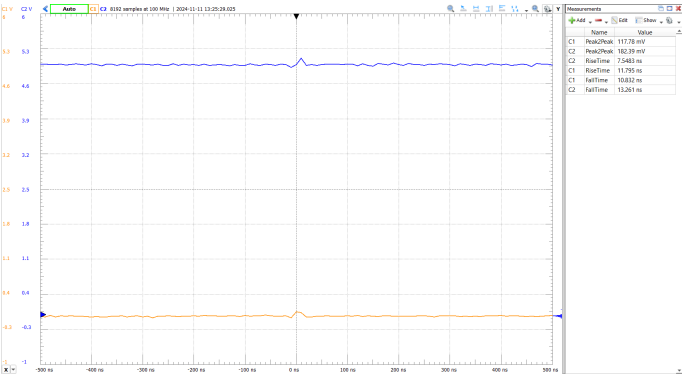
Figure 29. Noise induced (blue) on rising and falling edge of slammer signal (orange)

In fig. 29, we see that for the Golden Arduino, the power line has a p-p voltage of 3.7v and 3.4v on the rising and falling edge of the slammer signal, respectively.

12.2.2. Microcontroller as the Aggressor



(a) Noise on the rising edge of switch

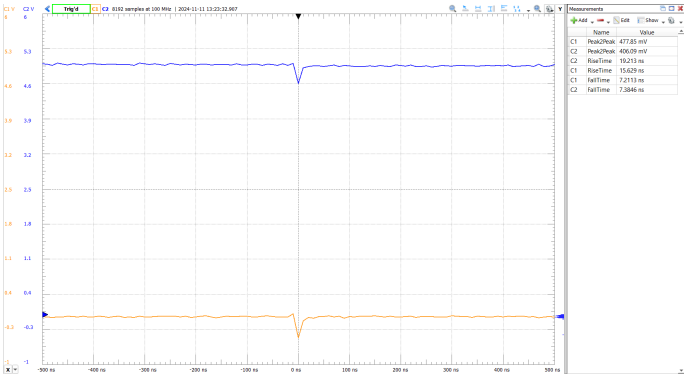


(b) Noise on the falling edge of switch

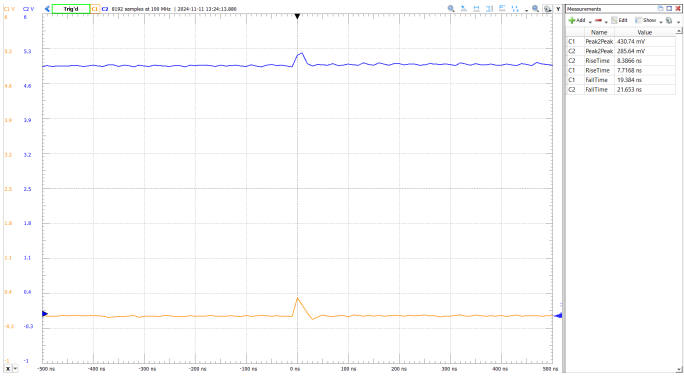
Figure 30. Noise induced on the HI (blue) and LO (orange) on the rising and falling edge of the switching signal (orange) with only pin 13 switching

Pin 13:

- 1. Rising Edge Vpp: .161v, .148v
- 2. Falling Edge Vpp: .118v, .182v



(a) Noise on the rising edge of switch

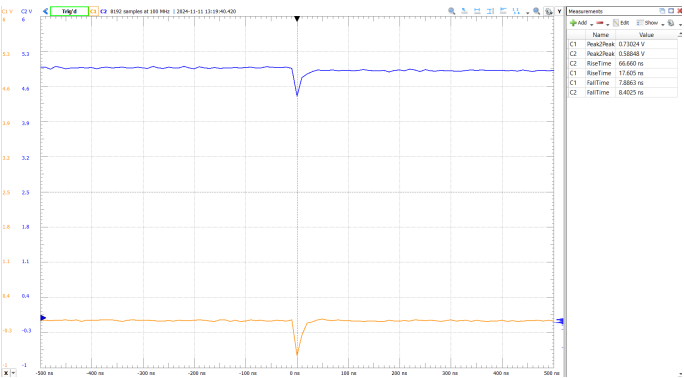


(b) Noise on the falling edge of switch

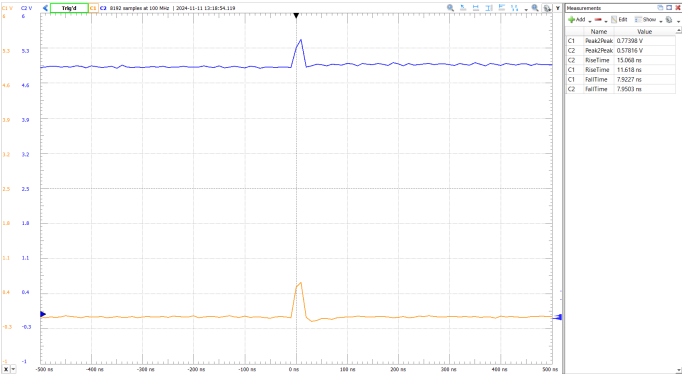
Figure 31. Noise induced on the HI (blue) and LO (orange) on the rising and falling edge of the switching signal (orange) with pins 13 and 12 switching

Pins 13 and 12:

- 1. Rising Edge Vpp: .478v, .406v
- 2. Falling Edge Vpp: .431v, .286v



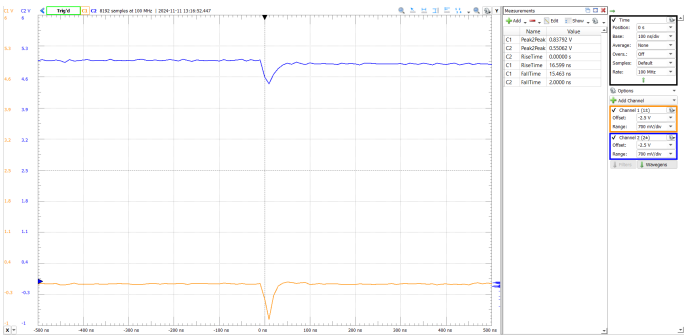
(a) Noise on the rising edge of switch



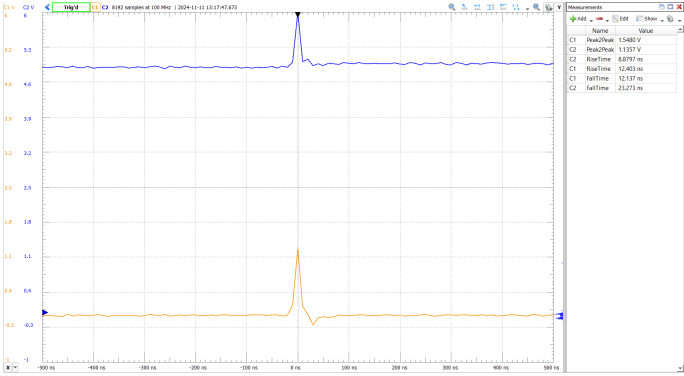
(b) Noise on the falling edge of switch

Figure 32. Noise induced on the HI (blue) and LO (orange) on the rising and falling edge of the switching signal (orange) with pins 13, 12, and 11 switching

- Pins 13, 12 and 11:
1. Rising Edge Vpp: .73v, .59v
 2. Falling Edge Vpp: .77v, .58v



(a) Noise on the rising edge of switch



(b) Noise on falling edge of switch

Figure 33. Noise induced on the HI (blue) and LO (orange) on the rising and falling edge of the switching signal (orange) with pins 13, 12, 11 and 10 switching

Pins 13, 12, 11 and 10

1. Rising Edge Vpp: .84v, .55v
2. Falling Edge Vpp: 1.55v, 1.13v

After completing the test, we see that as the number of switching pins increases, the noise on the HI and LO lines also increases.

12.3. Analysis

In the slammer circuit, both the Golden and commercial Arduinos performed the same. This implies that the Golden Arduino doesn't do much to reduce noise on the 5v rail. However, due to the jumper wires adding additional impedance, it cannot be concluded that either Arduino performed better.

In both the Golden and commercial Arduinos, the switching noise on the HI and LO lines progressively worsened as more pins switched simultaneously. However, we see that the Golden Arduino consistently reduced the noise by about .2v p-p, which demonstrates that it is better suited for faster switching signals.

It should be noted, however, that the Golden Arduino performed about the same when all pins were switching. This is, again, likely due to the jumper wires since we added a larger distance over which the current must travel. Since the board did not perform worse than the commercial one, we can still conclude that our board is an improvement over the commercial Arduino since it reduced the noise on the die for all other cases.

13. Conclusion

This lab aimed to create an improved version of the commercial Arduino by following the best design practices we've learned from previous labs. This can be broken down into two major steps: 1) Creating a fully functional Arduino and 2) proving it performs better than the commercial Arduino.

The Golden Arduino we made in this lab hits all of the criteria apart from a few soft errors (lack of TVS diode and incompatibility with the

switching noise shield), however, our Golden Arduino board is still fully functional and functions better than the commercial Arduino as it reduced the noise experience on the die. The following are key takeaways from this lab:

1. Not all boards are made with best design practices. Some manufacturers prioritize the cost of production over perfect signal integrity. Consequently, many boards may not be at their full potential. This should be taken into consideration when using boards in high-speed applications
2. For a more complex board, it is best to check for unexpected shorts after soldering a component. This saves a lot of headaches when powering on the board for testing as it ensures the board stays safe (assuming everything is routed correctly)
3. For a controller board, having test points on the data lines help in troubleshooting. If no test points are on these lines, debugging would be significantly harder since communication issues are hard to spot right away.
4. Always double-check the footprint and make sure it matches the part you are getting. The issue with the TVS diode was that there was a mismatch between the footprint on the board vs the actual diode. This caused some confusion on how the diode should be connected. By double-checking the footprint, we might've been able to include the TVS diode and create a more robust board.
5. Check for component spacing after designing the board. The spacing for the pins was fine when initially placed, however, they were moved during the designing process. By checking for component spacing at the end, we can ensure proper compatibility with other boards and reach a stronger conclusion about the performance of the Golden Arduino.